

Unsupervised Human Presence Detection from Indoor Sensor Signals: A Comparison of Classical, Feedforward, and Convolutional Anomaly Detection

Nur Taabaldiev

*Department of Computer Science, Sapienza University of Rome
AI Lab Course, Prof. D. Avola*

Abstract—Detecting whether an indoor space is occupied is useful for energy management, security, and smart-building automation. Most approaches treat this as a supervised classification problem that needs labeled examples of both empty and occupied periods. Labeled occupancy data is expensive and often unavailable in practice. This work studies the problem in a fully unsupervised, one-class setting: the models are trained only on data recorded while the room is empty, and human presence is detected as an anomaly at test time. We compare three methods that share the same data, the same time-based split, and the same threshold-selection procedure: an Isolation Forest using hand-crafted summary features, a feedforward Multilayer Perceptron (MLP) autoencoder, and a one-dimensional Convolutional Neural Network (CNN) autoencoder that reads the signal as an ordered sequence. Using the public UCI Occupancy Detection dataset with only the light and carbon-dioxide signals, we find that no single method dominates on every metric. The CNN autoencoder achieves the best F1 score (0.919) and the highest recall (0.988), while the Isolation Forest achieves the best ROC-AUC (0.928). The MLP is the weakest, which we trace to the loss of temporal structure caused by flattening the input window. We argue that the choice between these methods depends on whether missed detections or false alarms are more costly in the target application.

Index Terms—anomaly detection, unsupervised learning, autoencoder, convolutional neural network, occupancy detection, time-series.

I. Introduction

Knowing whether a room is occupied has many practical uses. Heating, ventilation, and lighting systems can save energy by reacting to real occupancy instead of fixed schedules. Security systems can flag presence in spaces that should be empty. Many of these settings cannot use cameras, either for privacy reasons or because cameras are too expensive to install everywhere. This motivates presence detection from cheap ambient sensors such as light, temperature, and carbon-dioxide (CO₂) meters.

The usual way to build such a detector is supervised learning: collect data labeled as empty or occupied, then train a classifier. The problem is that labeled occupancy data is costly to gather and rarely available for a new room. Collecting many hours of carefully labeled presence and absence is exactly the work we would like to avoid.

This paper takes a different view. We treat presence detection as an *anomaly detection* problem. The key idea is that an empty room is easy to observe: it is the default state, and recording it requires no human labeling effort beyond ensuring nobody is inside. If a model learns what an empty room looks like, then any period that does not fit that learned normal can be flagged as a possible presence. This is the one-class, or unsupervised, setting: training uses only normal (empty) data, and no occupied example is ever shown to the model during training.

Within this setting we ask a focused question: *which kind of model best captures the notion of a normal empty room, and does reading the temporal structure of the signal actually help?* We compare three methods of increasing structural awareness.

The first, Isolation Forest, is a classical method that works on hand-crafted summary statistics and ignores time order. The second, an MLP autoencoder, is a neural network that also ignores time order because it flattens the input. The third, a CNN autoencoder, processes the signal as an ordered sequence and can therefore react to local temporal patterns such as a sharp rise in CO₂.

Our contribution is not a new algorithm. It is a careful, fair comparison under a single experimental protocol, together with an analysis of why the methods behave differently and what that means for choosing one in practice. We deliberately keep the data split, the normalization, and the threshold rule identical across all three models so that any difference in performance can be attributed to the model itself.

II. Related Work

Occupancy detection from sensors. Environmental sensing for occupancy has been studied widely, often with the same UCI dataset used here. Candanedo and Feldheim [1] showed that light and CO₂ are strong predictors of occupancy and reported high accuracy with supervised models such as random forests and linear discriminant analysis. Most follow-up work also operates in the supervised regime and assumes labeled occupied periods are available.

One-class and unsupervised anomaly detection. When only normal data is available, anomaly detection methods learn a model of normality and score how far new samples deviate from it. Isolation Forest [2] isolates anomalies using random partitioning, under the intuition that anomalous points are easier

to separate from the rest. Density and distance based methods such as the Local Outlier Factor [3] take a different route. These methods are simple, fast, and require no neural network, which makes them strong baselines.

Autoencoders for anomaly detection. A reconstruction-based autoencoder is trained to copy normal inputs through a narrow bottleneck [4]. Because it only ever sees normal data, it reconstructs normal inputs well and abnormal inputs poorly, so the reconstruction error acts as an anomaly score. Convolutional and recurrent variants extend this idea to sequences so that temporal structure is preserved rather than discarded.

Position of this work. We bring these threads together in a single controlled study on indoor sensor signals, comparing a classical method, a feedforward autoencoder, and a convolutional autoencoder under identical conditions, and we analyze the trade-offs that result.

III. Dataset and Preprocessing

A. Dataset

We use the public UCI Occupancy Detection dataset [1]. It contains measurements from an office room sampled once per minute over roughly sixteen days. Each record has five signals (temperature, relative humidity, light, CO₂, and a derived humidity ratio) and a binary occupancy label. After cleaning, the dataset contains 20,560 valid records, of which about 77% are empty and 23% occupied.

The raw data, as distributed, stitches three separate files together. This left two stray header rows inside the data, where the date column literally contained the text *Temperature*. We removed these two rows and converted all columns to numeric types before any further processing.

B. Signal Selection

Figure 1 shows all five signals over time, with occupied periods shaded. Light and CO₂ react most clearly to presence: light jumps from near zero to several hundred lux when the room is in use, and CO₂ rises as people breathe and then falls slowly after they leave. Temperature responds weakly, and humidity barely responds at all. We therefore use only light and CO₂. Using the two most informative signals reduces noise and keeps the comparison between models clean.

C. Windowing

A single one-minute record carries little context, so we cut the signal into windows. Each window covers 30 consecutive minutes (30 records) of the two signals, giving a shape of 30×2 . Windows are taken with a stride of 5 minutes, so they overlap. A window is labeled occupied if the room was occupied during any minute inside it; otherwise it is empty. This produced 4,107 windows in total: 3,008 empty and 1,099 occupied.

D. Time-Based Split

Because the windows overlap, a random train and test split would be unsafe: two windows that share minutes could end up

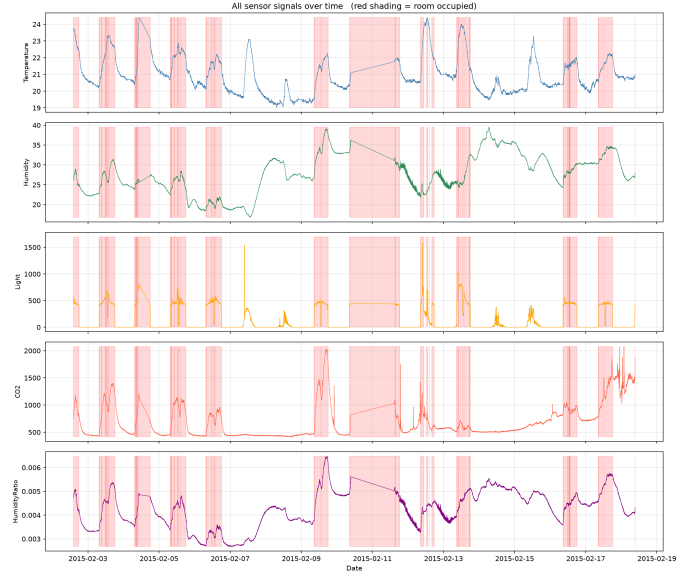


Figure 1: All five sensor signals over time. Red shading marks occupied periods. Light and CO₂ show the clearest response to presence, which is why they are the two signals used in this work.

on opposite sides of the split, leaking information from training into testing. To avoid this, we split in time order. Training and validation use empty windows only, so the models see nothing but normal data. The first 70% of empty windows (in time) form the training set, the next 15% form the validation set, and the final 15% are held out for testing. All occupied windows are placed in the test set. The resulting test set contains 452 empty and 1,099 occupied windows.

E. Normalization

Light and CO₂ live on very different numeric scales, so we apply z-score normalization to each signal. The mean and standard deviation are computed on the training set only and then applied to validation and test. Computing these statistics from training data alone prevents any information about the test distribution from leaking into the models.

IV. Methods

All three models follow the same one-class recipe: learn normality from empty windows, score each test window, and call a window occupied if its score is high enough. They differ in how they represent a window and how they produce the score.

A. Isolation Forest

Isolation Forest [2] cannot read a 30×2 window directly, so for each window we compute ten summary features: the mean, standard deviation, minimum, maximum, and range of each of the two signals. The forest is fitted on the features of the empty training windows. It repeatedly partitions the feature space with random splits; points that are isolated with few splits are scored as more anomalous. We use the negative of the model’s normality score so that a higher value always means more anomalous, matching the autoencoders.

B. MLP Autoencoder

An autoencoder compresses its input through a narrow bottleneck and then rebuilds it. Trained only on empty windows, it learns to reconstruct normal windows accurately. When it later receives an occupied window, which it has never seen, it reconstructs it poorly, and the reconstruction error becomes the anomaly score.

The MLP autoencoder first flattens the 30×2 window into a vector of 60 values. The encoder maps $60 \rightarrow 32 \rightarrow 16 \rightarrow 8$ and the decoder mirrors it as $8 \rightarrow 16 \rightarrow 32 \rightarrow 60$, with ReLU activations between layers. The bottleneck of size 8 is deliberately small; a bottleneck that is too large lets the network memorize inputs, which would make the reconstruction error useless as an anomaly signal. The loss is the mean squared error (MSE) between input and output. We train for 100 epochs with the Adam optimizer at a learning rate of 10^{-3} .

C. CNN Autoencoder

The CNN autoencoder keeps the window as an ordered sequence rather than flattening it. The input is arranged as two channels (light and CO_2) over 30 time steps. The encoder applies two one-dimensional convolutions followed by max-pooling; the decoder uses a transposed convolution and a final convolution to rebuild the sequence. A kernel size of 3 means each filter looks at three consecutive minutes at a time, so the model can respond to local temporal patterns such as a sharp rise in CO_2 over a few minutes. Training settings (empty windows only, MSE loss, Adam, 10^{-3} , 100 epochs) are identical to the MLP so that the only meaningful difference between the two networks is the architecture.

D. Threshold Selection and Metrics

For every model, the anomaly threshold is set to the 95th percentile of the scores on the validation set. Since validation contains only empty windows, this rule allows a small false-alarm rate on normal data while keeping the procedure identical across models. A test window is predicted occupied if its score exceeds the threshold.

We report four metrics. Precision is the fraction of windows flagged as occupied that are truly occupied. Recall is the fraction of truly occupied windows that are caught. F1 is the harmonic mean of precision and recall and depends on the chosen threshold. ROC-AUC measures how well the raw scores separate the two classes across all possible thresholds and is therefore threshold-independent. Reporting both F1 and ROC-AUC is important because, as the results show, they can disagree.

V. Experiments and Results

All randomness is seeded so that results reproduce exactly. The models are trained and evaluated on the splits described above.

A. Training Behavior

Both autoencoders train smoothly. Figure 2 and Figure 3 show that training and validation loss decrease together for the MLP and the CNN respectively, with no sign of overfitting; validation

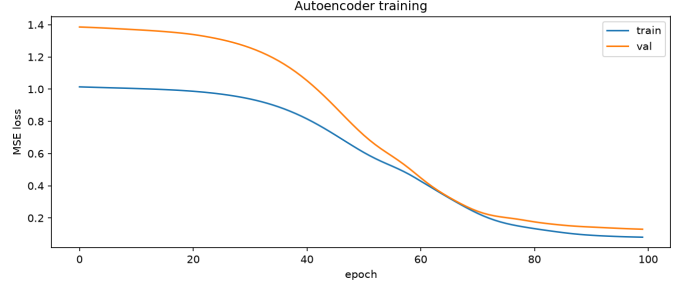


Figure 2: MLP autoencoder training. Training and validation MSE decrease together, indicating the model learned to reconstruct empty windows without overfitting.

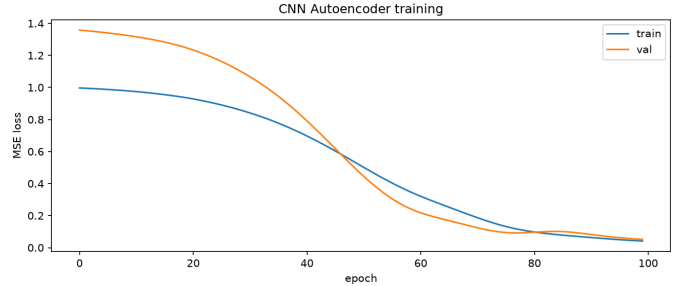


Figure 3: CNN autoencoder training. The curves track each other closely and settle at a low reconstruction error.

loss does not rise while training loss falls. This confirms that both networks successfully learned to reconstruct empty windows.

B. Score Distributions

A good anomaly detector should give empty and occupied windows clearly different scores. Figure 4 shows the Isolation Forest scores, where empty and occupied windows form two well-separated groups with only a narrow overlap. Figure 5 shows the MLP, where a large group of occupied windows has low reconstruction error and falls on the empty side of the threshold. Figure 6 shows the CNN, where occupied windows are pushed to higher errors and most cross the threshold, although the empty and occupied groups overlap more than for the Isolation Forest.

C. Quantitative Comparison

Table 1 reports all four metrics for the three models on the same test set. Figure 7 compares F1 and ROC-AUC side by side, and Figure 8 shows the ROC curves together.

Table 1: Detection results on the test set (452 empty, 1099 occupied windows). Best value in each column is in bold.

Model	Precision	Recall	F1	ROC-AUC
Isolation Forest	0.953	0.843	0.895	0.928
MLP Autoencoder	0.948	0.399	0.561	0.884
CNN Autoencoder	0.858	0.988	0.919	0.707

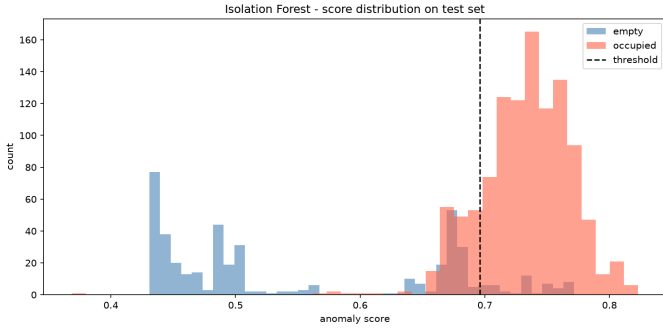


Figure 4: Isolation Forest score distribution on the test set. The two classes are well separated; the dashed line is the validation-based threshold.

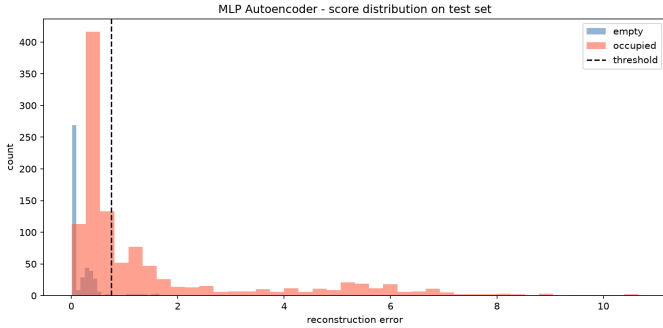


Figure 5: MLP autoencoder reconstruction-error distribution. Many occupied windows have low error and fall left of the threshold, which explains the low recall.

The confusion matrices give a more concrete picture. The CNN misses only 13 of the 1,099 occupied windows but raises 179 false alarms on the 452 empty windows. The Isolation Forest raises only 46 false alarms but misses 172 occupied windows. The MLP, despite high precision, misses 644 occupied windows.

VI. Discussion

No single winner. The most important observation is that the ranking of the models depends on the metric. The CNN wins on F1 and recall; the Isolation Forest wins on ROC-AUC. This is not a contradiction. F1 is computed at one chosen threshold, while ROC-AUC summarizes performance across all thresholds.

Why the MLP is weak. The MLP flattens each window into 60 unordered values. By doing so it discards the order of time. It can learn the typical *level* of each signal but not its *shape* over the 30 minutes. Many occupied windows happen to have average light and CO₂ values that resemble an empty room, for example at the very start of occupancy before CO₂ has accumulated. These windows are reconstructed almost as well as empty ones, so their error stays low and they slip below the threshold. This is the direct cause of the low recall of 0.399 seen in Figure 5.

Why the CNN helps but is not perfectly calibrated. The CNN keeps the time order and uses convolutional filters that respond to local change, so it reacts to the dynamics of occupancy

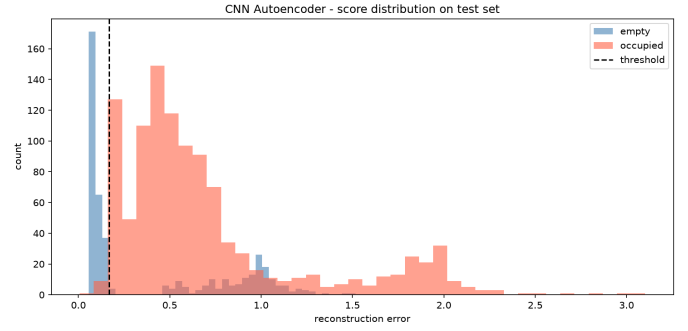


Figure 6: CNN autoencoder reconstruction-error distribution. Most occupied windows are pushed above the threshold, giving very high recall, at the cost of more false alarms.

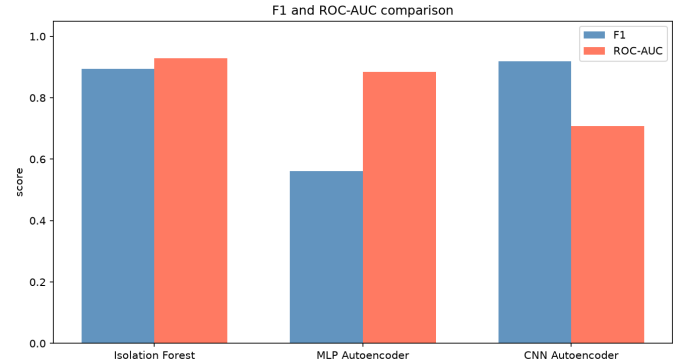


Figure 7: F1 and ROC-AUC for the three models. The CNN has the highest F1, the Isolation Forest the highest ROC-AUC, and the MLP is weakest on both.

rather than only the average level. This raises recall dramatically to 0.988. However, its raw scores are squeezed into a narrow range, which is why its ROC curve in Figure 8 stays flat and then jumps, and why its ROC-AUC is the lowest. In other words, the CNN is decisive at the chosen threshold but its scores are not as cleanly ordered as the Isolation Forest's.

Why the classical baseline remains strong. The Isolation Forest, using only ten hand-crafted features and no neural network, has the cleanest score separation of all three models. Its summary features (especially the mean and maximum of light and CO₂) already capture most of what distinguishes an occupied window. This is a useful reminder that a simple, well-chosen feature representation can match or beat a neural network on a problem of this size.

Practical implication. The right model depends on the cost of each kind of error. In a security setting, missing a present person is far worse than a false alarm, so the high recall of the CNN is preferable. In an energy-management setting, frequent false alarms would waste energy and annoy users, so the cleaner, more balanced Isolation Forest is the safer choice.

VII. Conclusion

We studied indoor human presence detection as an unsupervised, one-class problem, training only on empty-room data and detecting presence as an anomaly. Under a single fair proto-

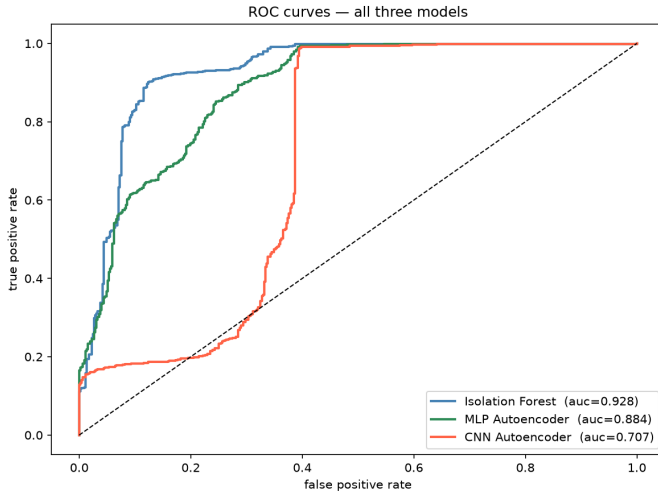


Figure 8: ROC curves for the three models. The Isolation Forest curve dominates across most of the range. The CNN curve stays low then jumps sharply, indicating that its scores are concentrated in a narrow band.

col, we compared an Isolation Forest, an MLP autoencoder, and a CNN autoencoder on the light and CO₂ signals of the UCI Occupancy Detection dataset. The CNN achieved the best F1 (0.919) and recall (0.988), the Isolation Forest the best ROC-AUC (0.928), and the MLP was weakest because flattening the window discards temporal structure. The main lesson is that respecting the time order of the signal improves detection, but that a simple classical baseline remains highly competitive and better calibrated.

Several directions remain. A Transformer encoder could be added as a fourth model to test whether self-attention captures normality better than convolution. Generalization could be tested by training on one room and evaluating on another. The fixed 95th-percentile threshold could be replaced by an adaptive rule. Finally, adding the weaker signals (temperature and humidity) would test whether they contribute useful information or only noise.

References

- [1] L. M. Candanedo and V. Feldheim, “Accurate occupancy detection of an office room from light, temperature, humidity and CO₂ measurements using statistical learning models,” *Energy and Buildings*, vol. 112, pp. 28–39, 2016.
- [2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [3] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: identifying density-based local outliers,” in *Proc. ACM SIGMOD*, 2000, pp. 93–104.
- [4] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proc. MLSDA Workshop*, 2014, pp. 4–11.
- [5] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 5998–6008.